

OS 4 Patterns Cheat Sheet

1. FORK + WAIT

Use when:

Create child processes and wait for them.

Syntax:

```
int f = fork();

if (f < 0) {
    perror("fork");
    exit(1);
} else if (f == 0) {
    // child
    exit(0);
} else {
    // parent
    wait(NULL);
}
```

Remember:

- `f == 0` in child
- `f > 0` in parent, and `f` is child's pid
- `wait(NULL)` waits for one child

2. PIPE + READ/WRITE

Use when:

Two processes need to send data.

Syntax:

```
int p[2];
pipe(p);

// p[0] = read end
// p[1] = write end
```

Example:

```
int x = 5;
write(p[1], &x, sizeof(int));
read(p[0], &x, sizeof(int));
```

With fork:

```
int p[2];
pipe(p);
int f = fork();

if (f == 0) {
    close(p[1]);
    int x;
    read(p[0], &x, sizeof(int));
    close(p[0]);
    exit(0);
} else {
    close(p[0]);
    int x = 5;
    write(p[1], &x, sizeof(int));
    close(p[1]);
    wait(NULL);
}
```

```
}
```

Remember:

- close unused ends
- one pipe = one direction
- two-way communication needs two pipes

```
=====
```

3. FIFO (NAMED PIPE)

```
=====
```

Use when:

Two separate programs need to send data.

Syntax:

```
mkfifo("myfifo", 0666);

int fd = open("myfifo", O_RDONLY);
read(fd, &x, sizeof(int));
close(fd);
```

Or:

```
int fd = open("myfifo", O_WRONLY);
write(fd, &x, sizeof(int));
close(fd);
```

Example:

```
// receiver
mkfifo("myfifo", 0666);
int fd = open("myfifo", O_RDONLY);
int x;
read(fd, &x, sizeof(int));
printf("%d\n", x);
close(fd);

// sender
int fd = open("myfifo", O_WRONLY);
int x = 5;
write(fd, &x, sizeof(int));
close(fd);
```

Remember:

- FIFO has a name in the filesystem
- use mkfifo before open
- useful for communication between separate programs
- remove it at the end with `unlink("myfifo")` if needed

```
=====
```

4. PTHREAD_CREATE + PTHREAD_JOIN

```
=====
```

Use when:

One process needs multiple threads.

Syntax:

```
void* f(void* arg) {
    return NULL;
}

pthread_t th;
pthread_create(&th, NULL, f, arg);
pthread_join(th, NULL);
```

Example:

```

void* f(void* arg) {
    int x = *(int*)arg;
    printf("%d\n", x);
    return NULL;
}

pthread_t th;
int x = 5;
pthread_create(&th, NULL, f, &x);
pthread_join(th, NULL);

```

Remember:

- thread function must be: void* f(void* arg)
- join waits for the thread
- all threads share memory

5. MUTEX LOCK/UNLOCK

Use when:

Multiple threads change the same variable.

Syntax:

```

pthread_mutex_t m;
pthread_mutex_init(&m, NULL);

pthread_mutex_lock(&m);
// critical section
pthread_mutex_unlock(&m);

pthread_mutex_destroy(&m);

```

Example:

```

int total = 0;

pthread_mutex_lock(&m);
total += value;
pthread_mutex_unlock(&m);

```

Why:

- without mutex, updates can overlap
- with mutex, one thread enters at a time

6. SEMAPHORE WAIT/POST

Use when:

You want to limit access or force an order between threads.

Syntax:

```

sem_t s;
sem_init(&s, 0, value);

sem_wait(&s);
// protected code
sem_post(&s);

sem_destroy(&s);

```

Example:

```

sem_t s;
sem_init(&s, 0, 1);

```

```
sem_wait(&s);
printf("one thread here\n");
sem_post(&s);
```

Remember:

- value 1 acts similar to a mutex
- sem_wait decreases the semaphore
- if value is 0, thread waits
- sem_post increases the semaphore

7. BARRIER

Use when:

Threads must all reach the same point before continuing.

Syntax:

```
pthread_barrier_t b;
pthread_barrier_init(&b, NULL, n);

pthread_barrier_wait(&b);

pthread_barrier_destroy(&b);
```

Example:

```
void* f(void* arg) {
    printf("before barrier\n");
    pthread_barrier_wait(&b);
    printf("after barrier\n");
    return NULL;
}
```

Remember:

- barrier count = number of threads that must arrive
- all threads wait there
- only after all arrive, they continue

FAST EXAM CHECKLIST

For fork problems:

- who is parent?
- who is child?
- does child exit?
- does parent wait?

For pipe problems:

- who sends to whom?
- one pipe per direction
- close unused ends
- read/write in the same order on both sides

For FIFO problems:

- who creates the FIFO?
- which program opens for read and which for write?
- does the FIFO need unlink at the end?

For thread problems:

- what argument does each thread receive?
- where does each thread store result?
- does main join all threads?

For mutex problems:

- what variable is shared?
- lock before changing it
- unlock after changing it

For semaphore problems:

- what is the initial value?
- does it protect access or enforce order?
- where are sem_wait and sem_post placed?

For barrier problems:

- how many threads must wait?
- where is the synchronization point?
- do all threads call pthread_barrier_wait?